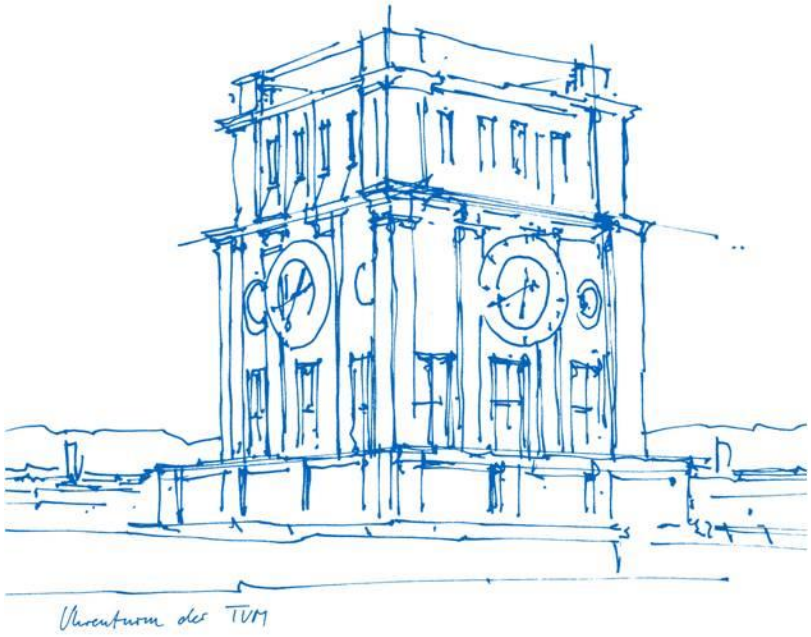
Sound B-tree for Fixed Size Keys/values & Comparing Retry Strategies for OLC

Mattias Puhk

Practical Course: Data Structure Engineering

Technische Universität München

05.02.2026



What has been built so far

- Concurrent B-Tree with Optimistic Lock Coupling
- Supports all standard operations
  - Search, insert, delete + iterator fns
- Two experimental variants
  - Result
  - Panic unwind
  - (default with inline restart)

Memory Architecture

- **BTree struct**
  - pages: Vec<Node>
  - root_id: AtomicU32
  - next_free_idx: AtomicUsize
  - entry_count: AtomicUsize
- **Node struct**
  - version: AtomicU64
  - keys: [K; 11]
  - values: [V; 11]
  - children: [NodeId; 12]
  - len: usize
  - is_leaf: bool

- **Why (default) capacity of 11 and branching factor 6?**
  - Balance tree height against Cache Line efficiency
  - Allows for a perfect symmetric split
  - With 8-byte keys -> 8 x 11 = 88 bytes
    - Span multiple cache lines

Design Decisions

- Pre-allocated Arena
  - Memory addresses do not change
  - If vector were to resize -> old references point to wrong things
- Raw pointers
  - Concurrent OLC fully without unsafe Rust?
  - Avoiding violations of Stacked Borrows
- No node recycling
  - Avoid ABA problem

Research Question

How does retry strategy affect OLC performance?

- We can have safety, but at what cost? Does rust's error handling add overhead?
- **Hypothesis**: Checking for errors at every step would add up.
- **Alternative**: Panic
- Which one performs better?

- Initial guesses:
  - Result + ? Operator adds branch overhead
  - Panic + catch_unwind has zero happy-path overhead
  - Branches cost cycles even when predicted correctly

Retry Strategy Comparison

Inline restart

```
'restart: loop {
    // ... traversal ...
    if !validate(v) {
        continue 'restart;
    }
}
```

Result + ?

```
fn inner() ->
  Result<T, VersionMismatch>
{
    // branch here
    validate(v)?;

    Ok(result)
}
```

Panic + Catch

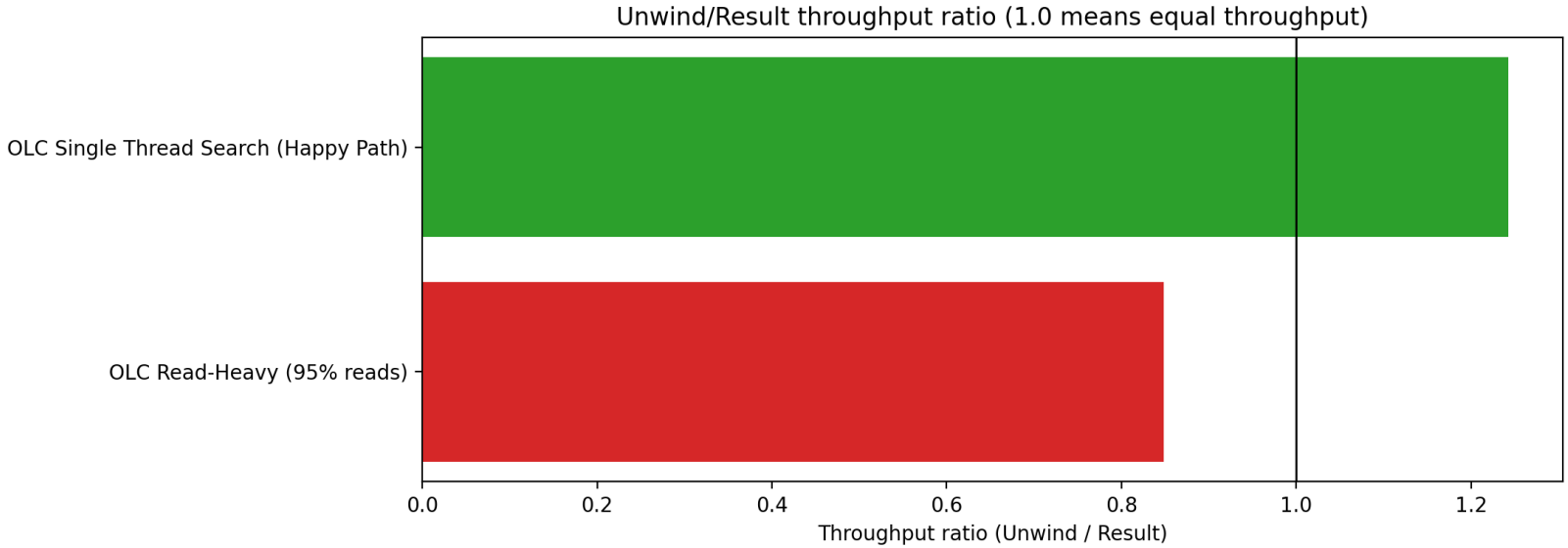
```
catch_unwind(|| {
    // no branch on happy path
    validate_or_panic(v);

    result
})
```

Benchmark Configuration

- Criterion.rs + Linux perf
  - Single-thread search (happy path)
  - Read-heavy concurrent
  - High contention

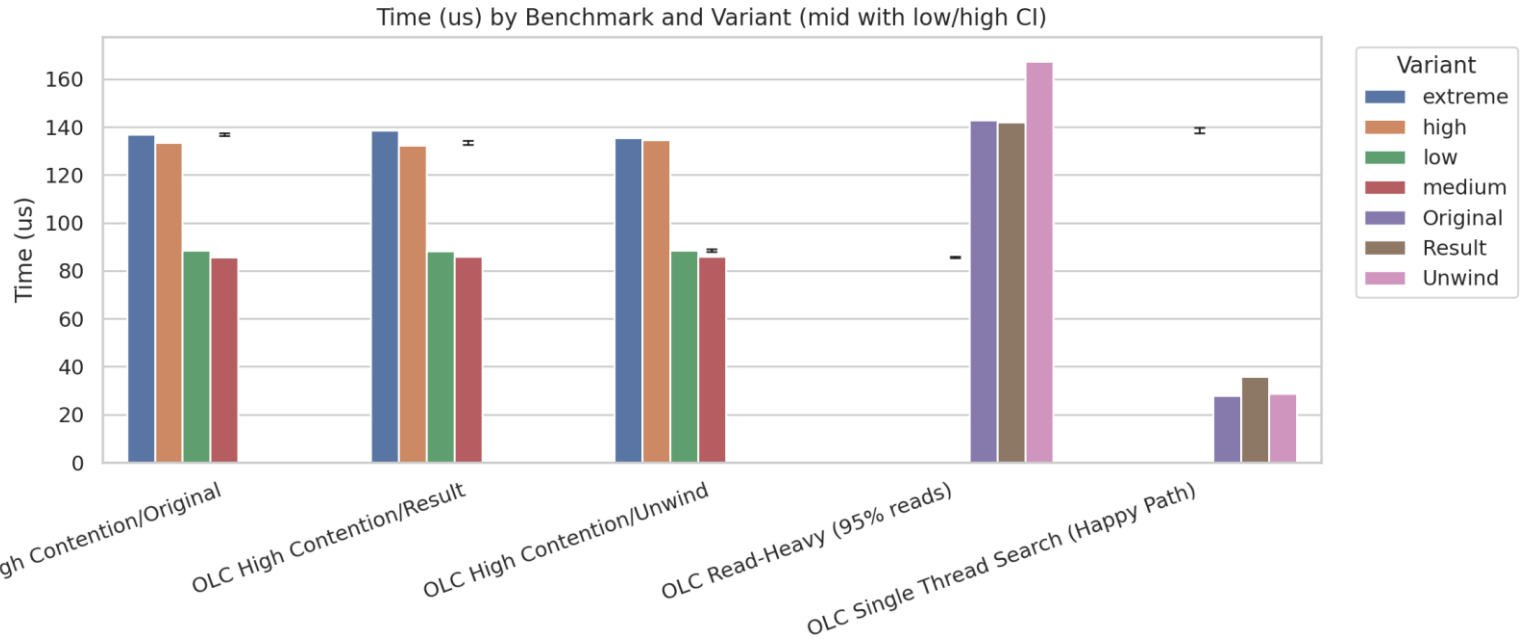
Single-Thread Search (Happy Path)

- Happy path -> Unwind performs better
- OLC Read-Heavy -> Result performs better



Concurrent Workload Results

- Read-Heavy (95% reads, 4 threads)
- High Contention (8 threads)



Trade-offs and Limitations

- **Benefit:** No global lock
  - Cost: More complex code & implementation
- **Benefit:** Thread-safe
  - Cost: 1.5-3x slower single-threaded

- **Limitations**
  - No node recycling
  - Fixed capacity at construction
  - Copy types only

Conclusions

- **Implementation**
  - Working concurrent B-tree with OLC
  - Safe public API (unsafe internals)
  - Arena provides robust memory management
- **Experimental Findings**
  - Result-based Retry -> +27% overhead
  - Unwind-based Retry matches inline performance
  - For rare-retry scenarios, unwind is a viable option
- **Potential Next Steps**
  - Node recycling (epoch-based reclamation?)
  - Range queries for efficient lookup

Thank you for your attention!